



Lecturer,
Dept. of CSE, ULAB, Dhaka



EMAIL:
atanu.shuvam@ulab.edu.bd

CSE 2201 : Algorithms

Lecture 14: Greedy Problems – Bin Packing

Atanu Shuvam Roy

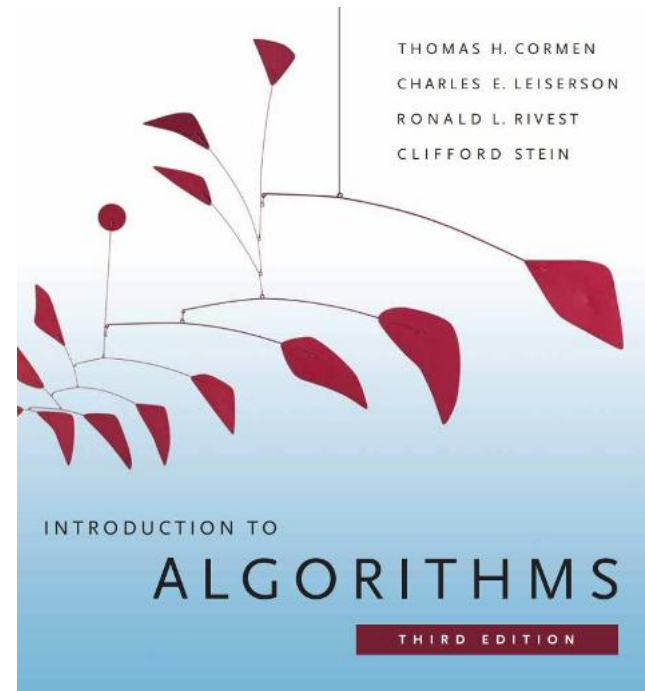


Class Code
GVWTHYJV



Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100

Today's Contents

- Introduction to Dynamic Programming
- Fibonacci Series
- Coin Change with Dynamic Programing

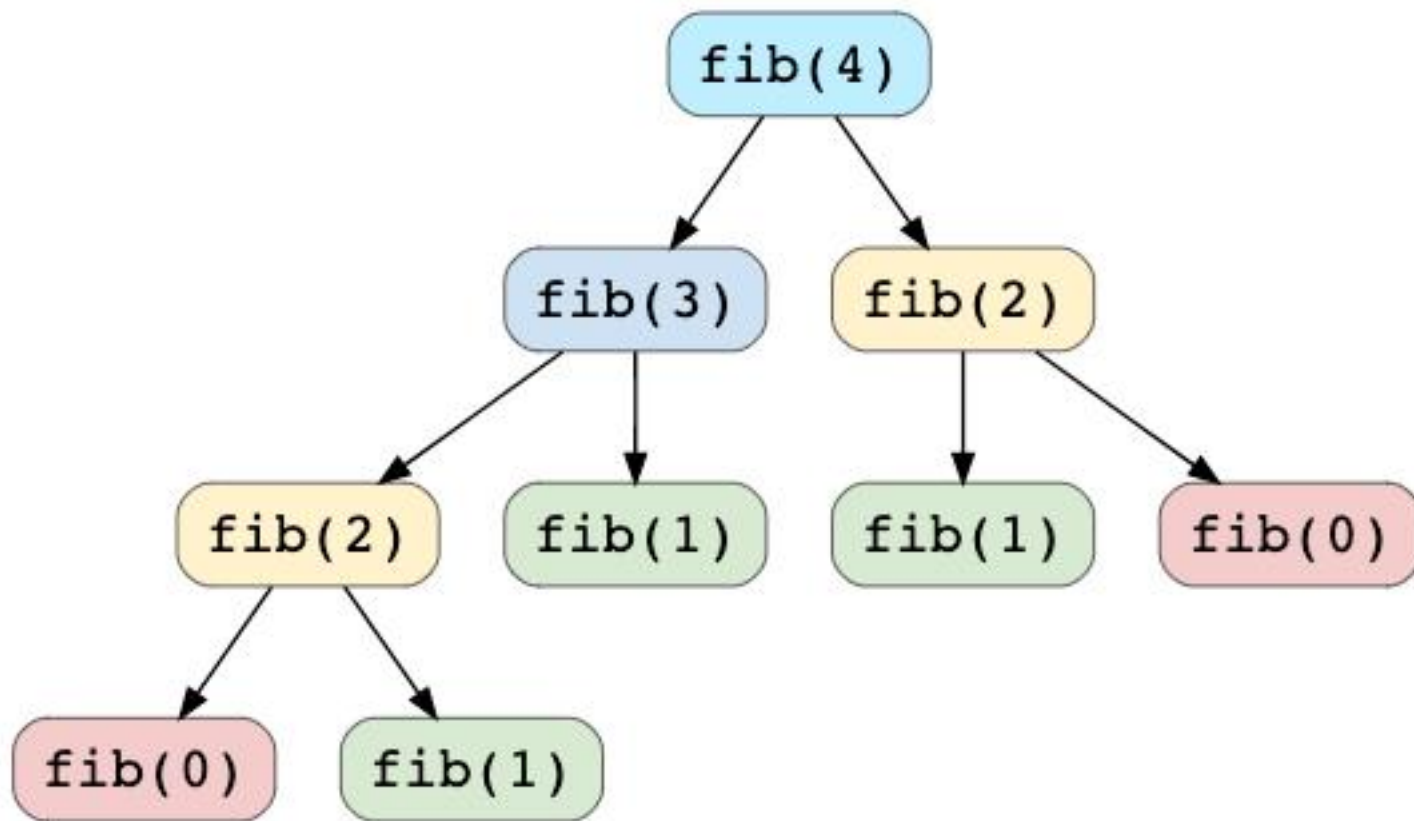
Dynamic Programming

- An algorithm design technique for **optimization problems** (similar to divide and conquer)
- Dynamic Programming **solves every subproblems just once and store the answer in a table**; next time when the result is needed , it simply use the data from the table.
- **Examines** the previously solved subproblems and **combines** their solutions to give the best solution for the whole problem.

Dynamic Programming

Part - 1 : Fibonacci Series

Fibonacci numbers



Fibonacci numbers

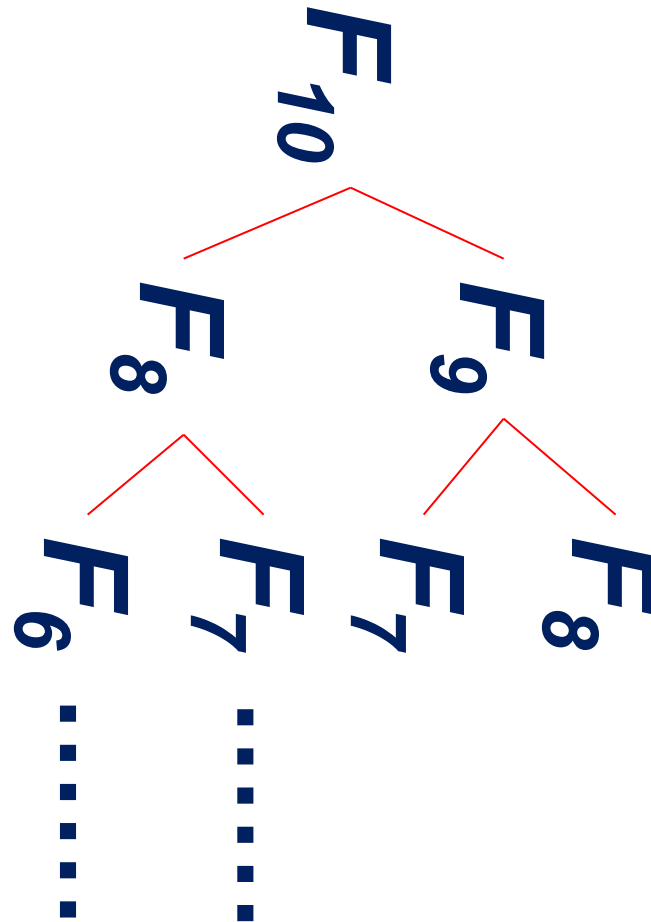
The *Fibonacci numbers* are defined by the following recurrence:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i > 1.$$

How to compute F_{10} ?



PseudoCode for Recursive Solution

```
int fibonacci (int n)
{
    if(n<=1)
        return n;

    return fibonacci(n-1) + fibonacci(n-2);
}
```

Dynamic Programming

- Applicable when subproblems are not independent
 - Subproblems share subsubproblems

E.g.: Fibonacci numbers:

- Recurrence: $F(n) = F(n-1) + F(n-2)$
 - A divide and conquer approach would repeatedly solve the common subproblems
 - **Dynamic programming solves every subproblem just once and stores the answer in a table**

Tabular computation F(n)

- The tabular computation can avoid recompilation.

Array $F[n+1]$

$F[0]=0;$

$F[1]=1;$

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}
0	1									

Result

$$F[i] = F[i-1] + F[i-2]$$

Tabular computation F(n)

- The tabular computation can avoid recompilation.

Array $F[n+1]$

$F[0]=0;$

$F[1]=1;$

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}
0	1	1	2	3	5	8	13	21	34	55

$$F[i] = F[i-1] + F[i-2]$$

Result

Psuodocode for DP solution

```
int fibonacci(int n)
{
    int F[n+1];

    F[0]=0;
    F[1]=1;

    for(i=2 to n)
        F[i]=F[i-1]+F[i-2];

    return F[n];
}
```

Reference

- <http://coursera.org/learn/algorithms-part1/>
- GeeksForGeeks
- YouTube
- LLMs
- Old Lectures of ULAB (Pramanik Sir)



Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh
Department of Computer Science & Engineering